

AUDITING STRUCTURED POLICY LEARNING FOR STREAMING TRANSACTION COLLATERAL CONTROL: FACTORIZATION, CONDITIONING, AND CROSS-SCALE TRANSFER

Yingda Yu

Affiliation to be confirmed

ABSTRACT

Streaming payment channels must route every arriving transaction into one of k collateral wallets or drop it, periodically flushing a wallet at a fixed fee. We study the resulting collateral-control problem as an online, partially-specified Markov decision process under twelve traffic regimes. We (i) faithfully port the environment/traffic generator and re-implement three PPO policies—a joint action head, independent factorized heads, and a settle-conditioned factored head. Across five seeds, the settle-conditioned factored head significantly beats the joint head at all four tested capacities, and the independently factored head at three of four, whereas the extra settle-conditioning path gives *no statistically detectable* gain (paired 95% CIs; zero- and shuffled-conditioning ablations are indistinguishable), a negative mechanism result we report. (ii) We introduce a permutation-equivariant set encoder with no k -dependent parameter, which loads and runs zero-shot across wallet counts where a flat MLP cannot; its transfer quality is asymmetric and it pays a modest matched-scale cost. (iii) A deliberately strong, validation-selected threshold rule attains the highest Money and significantly outperforms the learned policies in nearly all paired comparisons on stationary streams, and incurs zero avoidable drops after abrupt regime shifts. On this balanced workload, wallet fill is close to a sufficient statistic; we keep this negative rather than hide it. All numbers are produced by released, deterministic code on ported data.

Index Terms— reinforcement learning, payment channels, resource allocation, permutation equivariance, policy factorization, out-of-distribution.

1. INTRODUCTION

Off-chain payment networks such as the Lightning Network [1] process high-rate transaction streams over wallets backed by finite collateral. A controller observes the current transaction and wallet state, then either settles it into a wallet with sufficient free balance or drops it; it may also flush (empty and recharge) one wallet, incurring a fee and a short cooldown. Collateral C is partitioned into k wallets, so capacity is per-wallet, not shared: the controller must decide *where* to settle and *when/what* to flush under non-stationary, bursty traffic. Prior control schemes use fixed heuristics; recent work proposes learning the controller with policy optimization over a structured action space.

AI-assisted development disclosure: an AI coding assistant was used to re-implement code, run experiments, and draft text; all results are generated by the released code and data, and the author vets every claim. Author affiliation and funding to be confirmed.

We treat that line as a hypothesis to audit rather than a result to defend: *which structural inductive biases actually matter for on-line collateral control—a joint versus factored action head, a settle-conditioned flush head, a k -invariant encoder—and does learning beat a strong, cheap feedback rule?* Re-implementing the environment and policies, we benchmark against a validation-tuned rule (not only naive heuristics), add mechanism ablations, and redesign the encoder for the deployment reality that the number of wallets can change. The environment and twelve-regime generator are a faithful port of the prior manuscript; JA-PPO/IFAC/SC-FAC and their hyperparameters are re-implementations (the manuscript omits them); the strong rule, the conditioning ablations, and the equivariant set encoder are this study’s additions. Our findings are deliberately reported whether positive or null:

- A faithful, tested re-implementation of the environment and the twelve-regime traffic generator, and of three PPO policies—**JA-PPO** (joint action over $(k+1)^2$ outcomes), **IFAC** (independent settle and flush heads, $2(k+1)$ outputs), and **SC-FAC** (flush head conditioned on the chosen settle-ment wallet). We report the environment as a faithful port and the learned policies/hyperparameters as re-implementations.
- A **permutation-equivariant set encoder** (shared per-wallet function + pooling + per-wallet logits, in the spirit of Deep Sets [2]) with no parameter that depends on k , which transfers *zero-shot* across wallet counts; a flat multilayer perceptron cannot (its weight shapes change with k).
- **Mechanism ablations** that zero or shuffle the conditioning signal over matched seeds; they find no detectable Money benefit from the settle→flush path at the tested flush price—an honest null result.
- An **honest strong baseline**: a validation-selected best-fit threshold rule that attains zero avoidable drops, is robust to regime shifts, and attains the highest Money at every capacity, significantly beating the learned policies in nearly all paired comparisons. We report where learning does—and, more often, does not—beat it.

2. STREAMING COLLATERAL CONTROL

Model. Collateral $C \in \{800, 900, 1000, 1200\}$ is split into $k = 24$ wallets of capacity C/k . At each of $T = 1000$ decisions a transaction of size $x_t \in [0, 1000]$ arrives. The agent picks a settle wallet $a_s \in [0, k]$ and a flush wallet $a_f \in [0, k]$, where index k is a null (no-op). A flush executes first, emptying a wallet and making it unavailable for $F=3$ decisions before it refills to full; then the transaction is accepted if a_s is usable and has free balance $\geq x_t$. Drops are attributed as *oversize* ($x_t > C/k$, physically impossible), *frozen*

(cooldown), or *avoidable* (insufficient/conflict that a proactive flush could have prevented). The observation is $3k+2$: per-wallet normalized balance, availability, and cooldown, plus x_t normalized and time progress.

Objective. Money is $M = p \cdot \sum \text{accepted}_x - \tau \cdot \# \text{flushes}$ with $p = 1, \tau = 10$; episodic PPO [3] with GAE advantages [4] optimizes the policy. **Data.** Episodes are drawn from twelve regimes spanning steady/large transactions and burst traffic (calibrated to a target mean), with fixed seeds; 5000 training, 300 validation, and 200 test episodes per regime. All hyperparameters not given by the prior manuscript are fixed from validation pilots, never from test.

3. POLICIES

JA-PPO emits a categorical over all (a_s, a_f) pairs, i.e. $(k+1)^2=625$ logits at $k=24$, conflating the two decisions. **IFAC** factors the action into two independent $k+1$ categoricals (50 logits). **SC-FAC** factors but routes an embedding of the *chosen* settle wallet into the flush head, so flushing can depend on where value was just placed. Two ablations share its architecture: **no-cond.** zeroes the settle embedding, and **shuffled** feeds a deranged (wrong) wallet embedding.

Set encoder. The k wallets are an unordered set, yet a flat MLP treats wallet slots as a fixed-length ordered vector and ties its weights to k . We encode each wallet with a shared function $h_i = \phi(\text{balance}_i, \text{avail}_i, \text{cooldown}_i, g)$ of the wallet state and two global scalars, pool $\{h_i\}$, and produce per-wallet settle and flush logits. The map is equivariant to wallet permutations (proven by tests), and no layer’s size depends on k , so weights trained at one k load and run at another. The settle-conditioned set variant gathers the feature of the chosen settle wallet into the flush head.

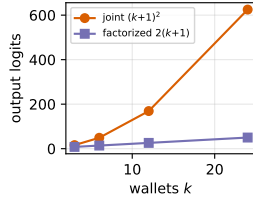


Fig. 1. Output logits vs. wallets k : joint $(k+1)^2$ vs. factorized $2(k+1)$.

policy	params	logits	latency (ms)
JA-PPO	245,874	625	0.42
IFAC	98,099	50	0.54
SC-FAC	172,883	50	0.62
SC-FAC (no cond.)	172,883	50	0.62
Set-IFAC	135,437	50	0.97
Set-SC-FAC	135,949	50	1.11

Table 1. Parameters, output logits, and CPU latency at $C=1200$, $k=24$. Set-encoder params are independent of k (transferable).

4. EXPERIMENTS

We evaluate deterministic policies on the fixed test pools. Model and training choices are made on validation only; we keep failed seeds and report negatives. *Baselines* are the native one-flush references

FA and FWF (reconstructed, since the reference uses a multi-flush design), a rotation rule, and the strong **BFP0.5**: best-fit routing that proactively flushes the fullest usable wallet once its fill crosses a threshold 0.5, chosen on validation. We also construct an oracle feasible value (transactions that fit some wallet).

Stationary performance. Table 2 reports Money per capacity with seed-level paired contrasts in Table 3; the structural floor of over-size drops is ≈ 400 per episode for every method. Two robust effects and one null emerge. *Factorization helps*: SC-FAC significantly outperforms JA-PPO at all four capacities ($p \leq 0.013$, 5/5 seeds); IFAC also outperforms JA-PPO at $C \in \{800, 900, 1200\}$ ($p \leq 0.033$) but *not* at $C=1000$ ($p=0.57$). *Conditioning does not add Money*: the SC-FAC–IFAC paired difference has a 95% CI including zero at $C \in \{900, 1000, 1200\}$, and at $C=800$ SC-FAC is significantly *lower* than IFAC (-419 , $p=0.009$); SC-FAC accepts marginally more value but spends more flushes, which cancels the gain at $\tau=10$. *A tuned rule beats learning*: BFP0.5 attains the highest Money at every capacity with zero avoidable drops, and its paired advantage is significant in 11/12 method-capacity contrasts; the sole exception is vs. IFAC at $C=1000$ ($+2545$, $p=0.057$, CI crossing zero), which BFP0.5 nevertheless wins on all five seeds. On this balanced i.i.d. workload a single balance-feedback threshold is near-optimal, and a 3000-episode PPO budget does not beat it.

method	$C = 800$	$C = 900$	$C = 1000$	$C = 1200$	paper rep.
FA	2907	4172	5533	8826	2773
FWF	2312	3482	4713	7640	2288
BFP0.5 (rule)	4682	7027	9497	15327	–
JA-PPO	3080 ± 144	4405 ± 185	6352 ± 36	11893 ± 149	3369
IFAC	4176 ± 69	5465 ± 188	6952 ± 960	13514 ± 96	3607
SC-FAC	3757 ± 43	5404 ± 104	7680 ± 45	13179 ± 259	3999

Table 2. Money per episode (mean $\pm$ SE over seeds) vs. collateral C . “paper rep.” is the prior manuscript’s reported value (not regenerated). Rules need no seeds.

contrast (a–b)	Δ Money	95% CI	p	wins/n
SC-FAC-IFAC	-336	[-1231,560]	0.357	1/5
IFAC-JA-PPO	1621	[1108,2134]	0.001	5/5
SC-FAC-JA-PPO	1285	[671,1900]	0.004	5/5
BFP0.5-SC-FAC	2148	[1428,2868]	0.001	5/5
BFP0.5-IFAC	1813	[1546,2079]	0.000	5/5
BFP0.5-JA-PPO	3433	[3020,3846]	0.000	5/5
SC-FAC-FA	4353	[3633,5072]	0.000	5/5
IFAC-FA	4688	[4421,4955]	0.000	5/5

Table 3. Seed-level paired Money differences at $C=1200$ (mean $a - b$, paired- t 95% CI, p -value, wins over seeds). Positive favors a .

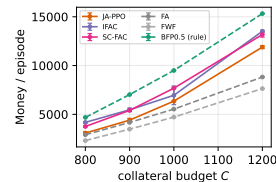


Fig. 2. Money vs. collateral C (solid: learned; dashed: rules).

Conditioning ablations. Table 4 compares SC-FAC against the no-conditioning and shuffled-conditioning variants under matched parameters, seeds, and budget. Neither zeroing nor shuffling the settle embedding lowers Money detectably: every paired 95% CI includes zero, and at $C=1200$ the fully conditioned variant is nominally the

lowest of the three. The settle→flush path therefore carries no detectable causal Money benefit at $\tau=10$ in this re-implementation.

variant	Money ($C=1200$, same 3 seeds)	flush cost
SC-FAC	12951±399	571
SC-FAC (no cond.)	13235±100	545
SC-FAC (shuf. cond.)	13252±217	579

Table 4. Settle-conditioning mechanism ablations ($C=1200$).

Cross- k transfer. We train at $k \in \{6, 12, 24\}$ and deploy at every k . Table 5 and Fig. 3 show that the set encoder loads and runs at every deployment k with no retraining, whereas flat policies have no off-diagonal entry because their state-dictionary shapes change with k . The benefit is operational deployability (train once, deploy at another scale), not uniformly preserved Money: zero-shot retention is asymmetric—strong when moving to larger-wallet/smaller- k operating points and weak when a model trained with large wallets meets the small wallets at $k=24$ —and the set encoder pays a modest matched- k cost versus a flat MLP.

train/deploy	k=6	k=12	k=24
$k = 6$	46742	26500	4433
$k = 12$	43733	36391	15172
$k = 24$	15435	15565	9190

Table 5. Money for Set-SC-FAC trained at rows, deployed at columns. Diagonal is matched training; off-diagonal is zero-shot cross- k transfer.

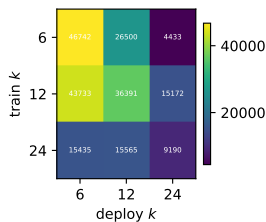


Fig. 3. Cross- k transfer matrix (Set-SC-FAC): rows train k , columns deploy k .

Non-stationary streams (OOD). Trained only on stationary pools, every policy is tested on the *same* deterministic episodes spliced across regimes at an abrupt change point (six scenarios: calm→burst, burst→calm, early and late). Table 6 and Fig. 4 report post-switch avoidable drops and Money. BFP0.5 incurs *zero* avoidable drops after every switch and the highest Money, adapting without retraining because wallet fill is close to a sufficient statistic. The learned policies generalize far better than the naive rules (28–56 vs. 144–148 post-switch drops) but remain significantly worse than BFP0.5 on both drops and Money over the paired scenarios ($p < 0.01$; $n=3$ seeds); IFAC and SC-FAC are tied. We therefore do *not* observe any switch-adaptation advantage of learning over the tuned feedback rule—an honest negative.

5. DISCUSSION AND LIMITATIONS

On balanced stationary streams a carefully tuned, extremely simple feedback rule attains the highest Money and beats the general-purpose PPO controllers in nearly all paired comparisons; the

policy	Money	post Money	post rec. drops
FA	8585	4232	143.8
FWF	7593	4075	148.4
BFP0.5 (rule)	15319	8206	0.0
JA-PPO	11694	6114	56.0
IFAC	13681	7220	28.3
SC-FAC	13108	7006	28.3

Table 6. Switching streams (mean over six scenarios; learned over three seeds). Post-switch avoidable drops and Money.

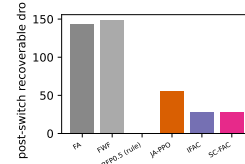


Fig. 4. Post-switch avoidable drops (mean over switch scenarios).

defensible claims for learning are therefore structural rather than raw Money. The robust learned effect is *factorization*: the settle-conditioned factored head beats the joint head at every capacity and the independent factored head at three of four; the further settle-conditioning mechanism is a null under our ablation protocol and should not be presented as a gain. The most useful systems result is the equivariant set encoder’s k -independence and zero-shot cross- k deployability, although we show this is asymmetric and trades off some matched-scale accuracy. We read the strong rule not as a failure of RL in general but as evidence that, with the present observation model, wallet fill is close to a sufficient statistic on i.i.d. streams; learning is most likely to matter only under richer structure (non-i.i.d. predictable bursts, heterogeneous fees/wallets), which we leave to future work. Limitations: one flush per step, finite horizon, a fixed fee, no lookahead to future arrivals, and single-GPU/CPU PPO with no vectorized rollouts. The prior manuscript’s learned numbers are treated as reported, not regenerated: its environment is a faithful port while the policies and unspecified hyperparameters are documented re-implementations chosen on validation.

6. CONCLUSION

We audited which structural inductive biases help a learned streaming collateral controller. The evidence is clear on three points and null on one. Factoring the joint action into separate settle/flush heads gives a reproducible gain over the joint head—significant at all four capacities for the settle-conditioned variant and at three of four for the independent variant—and cuts outputs from $(k+1)^2$ to $2(k+1)$. Adding a settle-conditioned flush path yields no detectable Money gain under matched-seed zero/shuffle ablations. A k -independent equivariant encoder adds genuine operational value—weights deploy zero-shot to unseen wallet counts where a flat MLP cannot—at an asymmetric transfer and matched-scale cost. Most importantly, on homogeneous stationary streams and across abrupt regime switches, one validation-selected feedback rule attains the highest Money with zero post-switch avoidable drops and beats the learned controllers in nearly every paired comparison; we present this as a target for future learning methods rather than conceal it.

7. REFERENCES

- [1] Joseph Poon and Thaddeus Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” Tech. Rep., Technical Report, 2016.
- [2] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander J. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [4] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.